

give me a start to finish steps breakdown in numbered form. just a summary of what i have to do

Here's the start-to-finish roadmap for Project 1. The goal is ultimately to produce a CSV with Name, State, Total Points, Pre-Rating, and Average Opponent Pre-Rating for every player.

1. Create your `.Rmd` file and put the Approach section you've already written at the top.
2. Load `tournamentinfo.txt` into R so the project works directly from the provided raw text file.
3. Inspect the raw text to understand its structure. Each player's information is mainly spread across two lines: one containing name/points/round results and another containing state/rating information.
4. `tournamentinfo.txt`TXT
5. Extract the basic player information from the text: pair number, player name, state, total points, and pre-tournament rating.
6. Clean the extracted information so things like spaces and special rating formats (`1641P17`, etc.) don't interfere with the numeric pre-rating.
7. Extract each player's opponent numbers from their round results. For example, Gary Hua's opponents are players 39, 21, 18, 14, 7, 12, and 4.
8. `tournamentinfo.txt`TXT
9. Handle rounds where there wasn't a normal opponent. Entries such as `H`, `B`, `U`, or `X` should not be treated as player numbers when calculating opponent ratings.
10. Match opponent numbers to player ratings. For example, if someone played opponent #39, R should look up player #39 and retrieve that player's pre-tournament rating.
11. Calculate each player's average opponent pre-rating using only the opponents they actually played. This is the main calculation of the project.
12. Build the final dataframe containing exactly the requested information: `Player Name | State | Total Points | Pre-Rating | Average Opponent Pre-Rating`.
13. Verify the results, especially Gary Hua. The assignment tells us his expected result is Gary Hua, ON, 6.0, 1794, 1605, so that's our first major accuracy check.
14. Export the final dataframe to `.csv` using R. That's the main output the project asks you to generate.
15. Clean up and knit the R Markdown, making sure someone else can run the code with access to the raw data.

16. Put the project/code/data on GitHub and publish the R Markdown to RPubS according to your course submission requirements. The Project 1 instructions specifically require the R Markdown to be published to RPubS and the data to be accessible to whoever runs the script.

So conceptually, the project is really just:

Messy TXT → extract players → extract opponents → match opponent ratings → calculate averages
→ clean dataframe → CSV.

The hardest part will be roughly steps 4–9. Once those work, the rest is straightforward.

11:05 PM

Next, extract the **opponent numbers** from each player's seven rounds. Add this chunk:

```
```{r}
round_results <- strsplit(player_lines, "|", fixed = TRUE)

opponent_numbers <- lapply(round_results, function(fields) {
 as.numeric(stringr::str_extract(fields[4:10], "\\d+"))
})

opponent_numbers[[1]]
```
```

`strsplit()` separates each line at the `|` characters. Fields 4–10 contain the seven round results, and `str_extract()` pulls out the opponent numbers. Rounds without an opponent become NA.

For Gary Hua, you should get:

```
[1] 39 21 18 14 7 12 4
```

Run this and check that those seven numbers match.

11:05 PM

yes i got all the players he played

11:08 PM

Now use those opponent numbers to look up their **pre-tournament ratings** and calculate the average for each player.

Add this chunk:

```
``{r}
players$avg_opponent_rating <- sapply(opponent_numbers, function(opponents) {
  opponent_ratings <- players$pre_rating[
    match(opponents, players$player_number)
  ]

  round(mean(opponent_ratings, na.rm = TRUE))
})
```

what does function do

`function(opponents)` defines a small reusable set of instructions for calculating **one player's average opponent rating**. Annotation 1

`sapply()` runs that function once for each player's list of opponents.

For Gary Hua, the steps are:

1. **Receive his opponent numbers** as `opponents`: 39, 21, 18, 14, 7, 12, 4.
2. **Find their rows** using `match(opponents, players$player_number)`.
3. **Get their ratings** from `players$pre_rating`.
4. **Calculate the average**, ignoring missing values with `na.rm = TRUE`, then round it.

The last expression inside the function is its result—in Gary's case, **1605**.

So this code means: “**For each player, look up their opponents' ratings, average them, and save the result in the new column.**”

```
head(players)
``
```

`match()` finds each opponent's row, and `mean(..., na.rm = TRUE)` averages their ratings while ignoring rounds without an opponent. `round()` gives a whole-number result.

You should now see **6 columns**, with **1605** as Gary Hua's `avg_opponent_rating`.

11:08 PM

yes

11:14 PM

it works

11:17 PM

You stopped after 2m 46s

Now create the final table with the **five required columns** and export it as a CSV.

Add this chunk:

```
``{r}
final_results <- data.frame(
  "Player Name" = players$player_name, "State" = players$state,
  "Total Points" = players$total_points, "Pre - Rating" = players$pre_rating,
  "Average Opponent Pre-Rating" = players$avg_opponent_rating,
  check.names = FALSE
)

knitr::kable(head(final_results))

write.csv(final_results, "chess_results.csv", row.names = FALSE)
``
```

The player number was needed to match opponents, but it isn't part of the required output.
`row.names = FALSE` prevents an extra row-number column in your CSV.

Run this, then click **Knit** to check that the entire document runs successfully. It should generate the HTML report and save `chess_results.csv` alongside your `.Rmd`.

11:17 PM

everything worked, got a clean csv file with new data